

Direct Linear Transform

Andrei Varga, 30433

Image Processing

The Problem

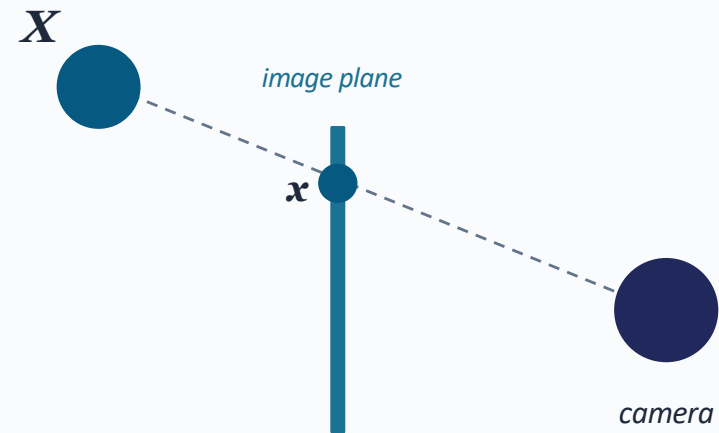
Camera calibration recovers the geometric model of how a camera maps 3D world points onto 2D image pixels.

Input

- An image of a known calibration object
- $N \geq 6$ correspondences (X_i, x_i) where X_i is 3D, x_i is 2D
- Constraint: world points must be non-coplanar

Output

A 3x4 projection matrix $P = K[R|t]$ — decomposable into intrinsics K and extrinsics (R, t) .



$$\lambda x = P \cdot X$$

Key Terms

P (3x4)

Projection matrix

Maps homogeneous 3D points to homogeneous 2D pixels.

K (3x3)

Intrinsics

Encodes focal length, principal point, pixel aspect, skew. Fixed by the camera hardware.

[R | t]

Extrinsics

Rotation R and translation t. Changes whenever the camera moves and describes camera position in the world.

λ

Depth

Scalar $\lambda x = PX$, the unknown depth where each ray meets the world point.

Related Work

Camera calibration is a well known and addressed problem.

1971



Abdel-Aziz & Karara

Original DLT formulation, from analytical photogrammetry

1987



R. Y. Tsai

Two-step calibration with a radial-distortion model

2000



Z. Zhang

Flexible calibration from planar grids — basis of modern checkerboard methods

2003



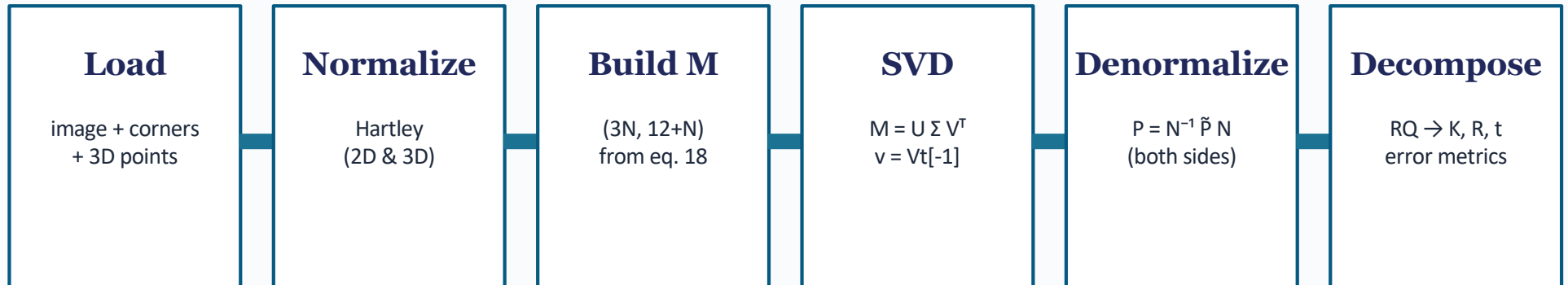
Hartley & Zisserman

Numerical normalization ($s = \sqrt{2}/\text{mean-dist}$) for stable DLT

Sources: Lecture 3(forelas3.pdf), Hartley & Zisserman, Multiple View Geometry (2003), Oxford VGG datasets

Pipeline

Six stages from raw correspondences to recovered P



Mathematical Foundation

$$\lambda_i \mathbf{x}_i = \mathbf{P} \cdot \mathbf{X}_i$$

From parallel vectors to a homogeneous linear system

1. Eliminate the depths λ_i via the cross product: $\mathbf{x}_i \times \mathbf{P} \mathbf{X}_i = \mathbf{0}$.
2. Each correspondence contributes 2 independent linear equations in the 12 unknowns of $\mathbf{P}$.
3. Stack rows from $N \geq 6$ points into $\mathbf{M}$; solve ***min $\|\mathbf{M}\mathbf{v}\|^2$ subject to $\|\mathbf{v}\| = 1$*** using Singular Value Decomposition.
4. The solution is the right-singular vector of $\mathbf{M}$ with the smallest singular value.

The M Matrix

Per-point block: 3 rows per correspondence

$\mathbf{Xi}^T$	0	0	$-x_i \text{ at } \lambda_i \text{ col}$
0	$\mathbf{Xi}^T$	0	$-y_i \text{ at } \lambda_i \text{ col}$
0	0	$\mathbf{Xi}^T$	$-1 \text{ at } \lambda_i \text{ col}$

Block structure

$\mathbf{Xi}^T = (X, Y, Z, 1)$

0 (1x4 placeholder)

depth col: $-x_i, -y_i, -1$

Full matrix shape: $(3N, 12 + N)$ — 12 unknowns for the entries of P, plus one λ_i depth per point. After SVD, the first 12 entries reshape into the 3x4 P.

Hartley Normalization

Preconditioning the SVD for numerical stability

Problem

M contains pixel coordinates ($\sim 10^3$) mixed with homogeneous 1s. Columns differ by $\sim 10^3$ in magnitude, so $M^T M$ is poorly conditioned and singular value decomposition can give unpredictable results.

Solution

Translate to centroid, then scale so the average point sits at distance **sqrt(2) (2D)** or **sqrt(3) (3D)**. Build M from normalized points, solve, then map back.

$$P = N_{img}^{-1} \cdot P^* \cdot N_{world}$$

SVD Solution

$$\mathbf{M} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

1. Solve $\min \|\mathbf{M}\mathbf{v}\|^2$ subject to $\|\mathbf{v}\| = 1$, the one chosen is the singular vector corresponding with the smallest singular value.
2. NumPy returns $\mathbf{V}^T$ with singular values in descending order, so the answer is $\mathbf{v} = \mathbf{V}^t[-1]$, the last row.
3. Reshape the first 12 entries: $\mathbf{P} = \mathbf{v}[:12].\text{reshape}(3, 4)$. The remaining entries are the per-point depths λ_i .

Implementation

<code>normalize_2d(pts)</code>	→ Hartley normalization, 3x3 matrix (image)
<code>normalize_3d(pts)</code>	→ Hartley normalization, 4x4 matrix (world)
<code>build_M_matrix(x, X)</code>	→ stack 3N rows, returns (3N, 12+N)
<code>get_dlt_solution(M)</code>	→ SVD, returns smallest right-singular vector
<code>solve_resection(x, X)</code>	→ full pipeline: normalize → build → SVD → denormalize
<code>reprojection_error(P,..)</code>	→ project X_i through P , measure pixel distance to x_i
<code>get_inner_params(P)</code>	→ RQ-factorize the 3x3 left block → K, R, t
<code>show_projections(...)</code>	→ OpenCV side-by-side visualization

Running the Project

1

Install dependencies

```
pip install numpy opencv-python
```

2

Point to the dataset

```
export IP_PROJECT_DATA="/path/to/data"
```

3

Run

```
python dlt1.py
```

Requirements: download the House and Corridor datasets from <https://www.robots.ox.ac.uk/~vgg/data/mview/>

Test Datasets

Both come from the Oxford VGG Multi-View collection.

Model House

10 views · ~672 reconstructed 3D points

Wireframe-like building with sharp corners on multiple depth planes.

Corridor

11 views · ~330 reconstructed 3D points

Long interior view with strong perspective foreshortening. Tests numerical stability when depths vary by orders of magnitude.

Conclusions



DLT recovers P in closed form

A single SVD on the stacked system yields the projection matrix without iteration.



Linear is a starting point

DLT provides the right initial estimate for nonlinear refinement.